

★★ 3. If Exp == 0

Overview

This documentation describes the implementation of an `if exp == 0` expression form in a typed object-oriented language. The implementation includes the addition of conditional expressions with type checking that ensures type soundness.

Implementation Details

1. Syntax Addition

Added `ifzeroI` to represent conditional expressions in the abstract syntax:

```
| ifzeroI(test :: ExpI,  
          then :: ExpI,  
          else :: ExpI)
```

This syntax allows expressions of the form:

```
if <test-exp> == 0 | <then-exp> | <else-exp>
```

2. Type Checking Implementation

The type checking for `ifzeroI` is implemented in the `typecheck_exp` function with the following key requirements:

1. The test expression must be an integer type:

```
| ifzeroI(test, then, else):  
  match recur(test)  
  | intT():  
    def then_type = recur(then)  
    def else_type = recur(else)  
    cond
```

```

    | then_type == else_type: then_type
    | is_subtype(then_type, else_type, t_classes): else_t
ype
    | is_subtype(else_type, then_type, t_classes): then_t
ype
    | ~else: type_error(exp, "no common supertype")
  | ~else:
    type_error(test, "Int")

```

Key design decisions in type checking:

- The test expression must evaluate to an `intT()`
- The then and else branches can have different types as long as they share a common supertype
- The result type is determined by finding the least upper bound of the branch types

3. Interpreter Implementation

The interpreter implementation in `class.rhm` handles the runtime evaluation:

```

| ifzeroE(test, then, else):
  match recur(test)
  | intV(n):
    cond
    | n == 0: recur(then)
    | ~else: recur(else)
  | ~else:
    error('#interp, "condition must be number")

```

Key features:

- Evaluates the test expression first
- Only proceeds if test produces a number value
- Executes appropriate branch based on test result being zero

4. Type Soundness

The implementation maintains type soundness through several key properties:

1. **Type Preservation:** The type checker ensures that if an expression has type T , its evaluation produces a value of type T or a subtype of T .
2. **Progress:** Well-typed expressions cannot get stuck - they either evaluate to a value or raise a type error.
3. **Subtype Consistency:** The type system properly handles subtyping relationships when determining the result type of conditional expressions.

Testing

The implementation includes comprehensive test cases:

```
check: typecheck(ifzeroI(intI(0),
                        newI('#Posn3D, [intI(1), intI(2), int
I(3)]),
                        newI('#Posn, [intI(1), intI(2)])),
        [posn_t_class, posn3D_t_class])
~is objT('#Posn)
```

Test cases cover:

1. Basic functionality with same-typed branches
2. Subtype relationships in branch expressions
3. Error cases for non-integer test expressions
4. Complex object type hierarchies

Design Choices and Rationale

1. Type Inference Strategy

- Chose to find the least upper bound of branch types
- This provides maximum flexibility while maintaining type safety
- Allows natural expression of polymorphic conditionals

2. Error Handling

- Strict checking of test expression type
- Clear error messages for type mismatches
- Runtime validation of numeric values

3. Subtyping Relations

- Leveraged existing subtype checking mechanism
- Maintained consistency with object-oriented type hierarchy

Conclusion

The implementation successfully adds conditional expressions to the language while maintaining type safety and providing clear semantics for type checking and evaluation. The design choices prioritize both flexibility and soundness, allowing for expressive conditional statements within the type system's constraints.